Czech Bank Dormant Account Analysis

A SQL analysis in MySQL Workbench

Oct 21, 2026 · @Nani

1. Problem Statement

A retail bank can only act on a customer who is going quiet if it notices early enough. Most accounts don't close. They stop being used, and by the time that shows up, the customer has usually already moved their activity elsewhere. Spotting these accounts early gives the bank a chance to step in before they are lost.

The Czech Bank data has no churn label, so there is no column saying which accounts have gone dormant. The first problem is to define dormancy from the transactions themselves, and the definition has to hold up. A bank also posts interest, statement fees and sanction interest to accounts on its own, so those entries can't count as customer activity. The second problem is to find out what dormant accounts looked like in the months before they went quiet, so the next ones can be flagged sooner.

2. Objective

1. **Understand the database.** Study the structure of the Czech Bank database and how its tables relate, and draw an entity relationship diagram.
2. **Clean and transform the data.** Prepare the tables for analysis in MySQL. This covers converting dates, translating coded values to English, and turning blank values into NULL.
3. **Identify dormant accounts.** Define dormancy from customer activity only, excluding interest, statement fees and sanction interest. Flag accounts with no customer activity for more than 90 days as of 31 December 1998.
4. **Analyse their behaviour.** Study how dormant accounts behaved before going quiet, and compare them with active accounts.

3. Database and Understanding

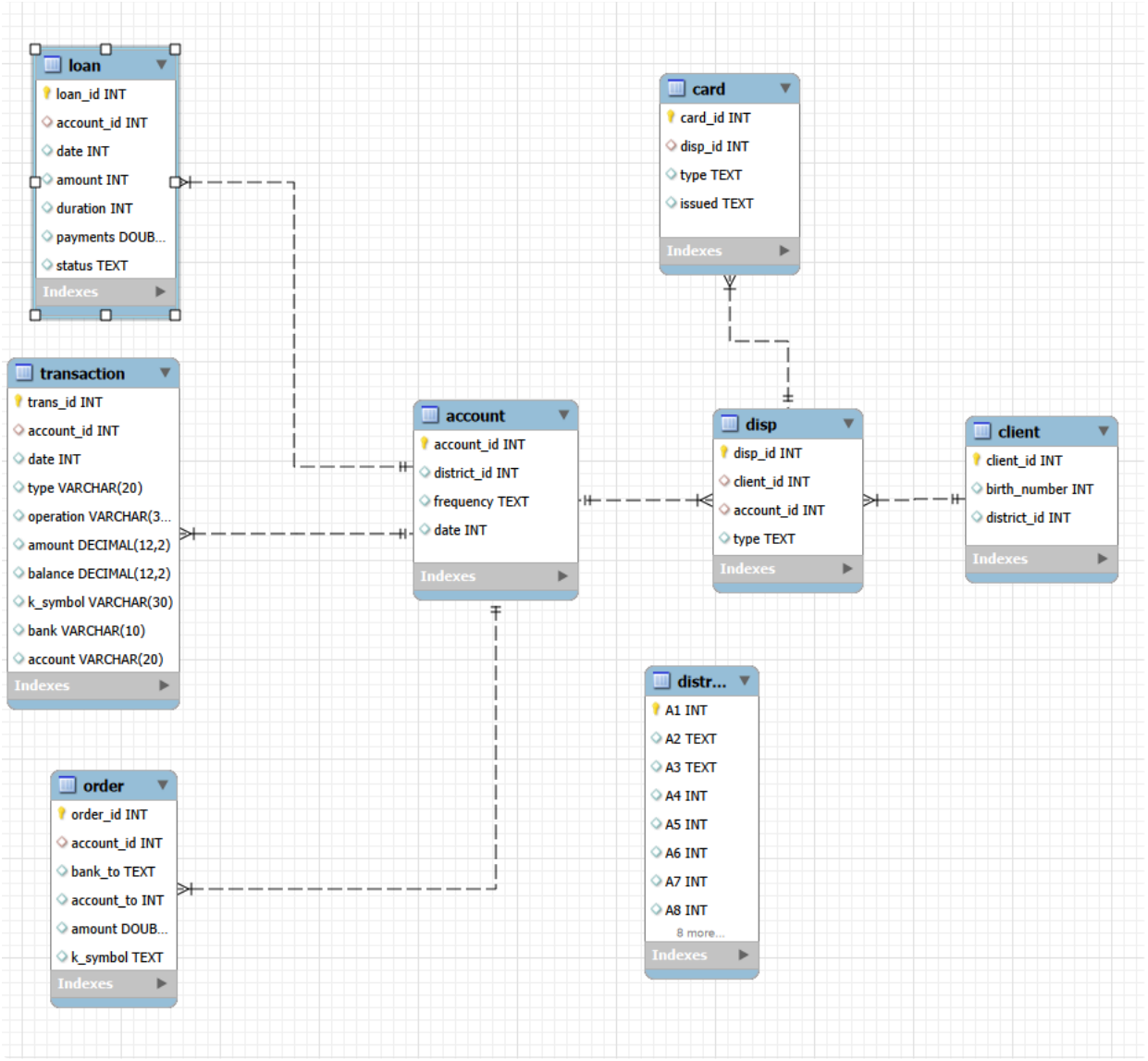
3.1 About the data

The dataset is a collection of financial information from a Czech bank. It covers over 5,300 bank clients and approximately 1,000,000 transactions. The bank in the dataset has extended close to 700 loans and issued nearly 900 credit cards, and all of these are

represented in the data.

3.2 Entity relationships

- Each account has static characteristics (date of creation, address of the branch) in the `account` table, and dynamic characteristics (payments debited or credited, balances) in the `order` and `trans` tables.
- The `client` table describes the persons who can manipulate the accounts.
- One client can have more than one account, and more than one client can manipulate a single account. The `disp` (disposition) table relates clients and accounts.
- The `loan` and `card` tables describe services the bank offers to its clients.
- More than one credit card can be issued to an account, but at most one loan can be granted for an account.
- The `district` table gives publicly available information about the districts (for example, the unemployment rate). Additional information about the clients can be deduced from it.



3.3 Table descriptions

Table	Description	Records
`account`	Each record describes the static characteristics of an account.	4,500
`client`	Each record describes the characteristics of a client.	5,369
`disp` (disposition)	Each record relates a client to an account, i.e. it describes the rights of clients to operate accounts.	5,369
`order` (permanent orders, debits only)	Each record describes the characteristics of a payment order.	6,471
`trans` (transactions)	Each record describes one transaction on an account.	1,056,320
`loan`	Each record describes a loan granted for a given account.	682
`card` (credit cards)	Each record describes a credit card issued to an account.	892
`district` (demographic data)	Each record describes the demographic characteristics of a district.	77

3.4 Table schemas

account

Column	Description	Notes
`account_id`	Identification of the account	
`district_id`	Location of the branch	
`date`	Date of the account's creation	In the form YYMMDD
`frequency`	Frequency of statement issuance	"POPLATEK MESICNE" = Monthly issuance; "POPLATEK TYDNE" = Weekly issuance; "POPLATEK PO OBRATU" = Issuance after transaction

client

Column	Description	Notes
`client_id`	Client identifier	
`birth_number`	Birthday and sex	YYMMDD for men; YYMM+50DD for women, where YYMMDD is the date of birth
`district_id`	Address of the client	

disp

Column	Description	Notes
`disp_id`	Record identifier	
`client_id`	Client identifier	
`account_id`	Account identifier	
`type`	Type of disposition (owner/user)	Only the owner can issue permanent orders and ask for a loan

loan

Column	Description	Notes
`disp_id`	Record identifier	
`loan_id`	Record identifier	
`account_id`	Account identifier	
`date`	Date when the loan was granted	In the form YYMMDD
`amount`	Amount of the loan	
`duration`	Duration of the loan	
`payments`	Monthly payments on the loan	

Column	Description	Notes
`status`	Status in paying off the loan	'A' = contract finished, no problems; 'B' = contract finished, loan not paid; 'C' = running contract, OK so far; 'D' = running contract, client in debt

order

Column	Description	Notes
`order_id`	Record identifier	
`account_id`	Account the order is issued for	
`bank_to`	Bank of the recipient	Each bank has a unique two-letter code
`account_to`	Account of the recipient	
`amount`	Amount debited from the order account	
`k_symbol`	Characterization of the payment	'POJISTNE' = Insurance payment; 'SIPO' = Household payment; 'LEASING' = Leasing payment; 'UVER' = Loan payment

trans

Column	Description	Notes
`trans_id`	Record identifier	
`account_id`	Account the transaction is issued on	
`date`	Date of transaction	In the form YYMMDD
`type`	Debit/credit transaction	'PRIJEM' = Credit; 'VYDAJ' = Debit (withdrawal)
`operation`	Mode of transaction	'VYBER KARTOU' = Credit card withdrawal; 'VKLAD' = Credit in cash; 'PREVOD Z UCTU' = Collection from another bank; 'VYBER' = Withdrawal in cash; 'PREVOD NA UCET' = Remittance to another bank

Column	Description	Notes
`amount`	Amount of transaction	
`balance`	Balance of account after transaction	
`k_symbol`	Characterization of transaction	'POJISTNE' = Insurance payment; 'SLUZBY' = Payment for statement; 'UROK' = Interest credited; 'SANKC. UROK' = Sanction interest if negative balance; 'SIPO' = Household payment; 'DUCHOD' = Old-age pension payment; 'UVER' = Loan payment
`bank`	Bank of the partner	Each bank has a unique two-letter code
`account`	Account of the partner	

card

Column	Description	Notes
`card_id`	Card identifier	
`disp_id`	Disposition to an account	
`type`	Type of card	'Junior', 'Classic' or 'Gold'
`issued`	Date the card was issued	In the format YYMMDD

district (demographic data)

Column	Description
`A1` (`district_id`)	District identifier
`A2`	District name
`A3`	Region
`A4`	No. of inhabitants
`A5`	No. of municipalities with inhabitants < 499
`A6`	No. of municipalities with inhabitants 500-1999
`A7`	No. of municipalities with inhabitants 2000-9999

Column	Description
`A8`	No. of municipalities with inhabitants > 10000
`A9`	No. of cities
`A10`	Ratio of urban inhabitants
`A11`	Average salary
`A12`	Unemployment rate in 1995
`A13`	Unemployment rate in 1996
`A14`	No. of entrepreneurs per 1000 inhabitants
`A15`	No. of crimes committed in 1995
`A16`	No. of crimes committed in 1996

4. Methodology

4.1 Data loading

The database `czech_bank` was created in MySQL Workbench, and the tables were loaded in two ways:

- **Seven tables** (`account`, `client`, `disp`, `card`, `loan`, `order`, `district`) were loaded with the Table Data Import Wizard.
- **`trans`** was loaded differently to make it faster, since it holds over a million rows. The table was created with defined data types, then filled with `LOAD DATA LOCAL INFILE` from the semicolon-separated `trans.csv`.

The first `LOAD DATA LOCAL INFILE` attempt failed with error 2068. To fix it, `OPT_LOCAL_INFILE=1` was set in the connection's Advanced settings in Workbench. The server's `local_infile` setting was already ON. After loading, a row count (1,056,320) and a preview of the first rows confirmed the data was in. The distinct values of `type`, `operation` and `k_symbol` were listed to see which codes needed translating.

4.2 Data cleaning and transformation

The raw `trans` table was left untouched. A new table, `trans_clean`, was created from it with these changes:

Change	Detail
Date	`date` (YYMMDD number) converted to a proper date, `trans_date`
Blank values	Text blanks trimmed and set to NULL
English labels	Added `type_en`, `operation_en` and `k_symbol_en` for the Czech codes

The other tables were cleaned in place:

Table	Change
`account`	`date` converted to `created_date` and the old column dropped. Added `frequency_en` with English statement-frequency labels.
`client`	Added `birth_date`, decoded from `birth_number`. For women the month has 50 added, so 50 is subtracted.
`card`	`issued` converted to a datetime.
`order`	Added `k_symbol_en` with English labels, and blanks set to NULL.

`loan`, `disp` and `district` were viewed but not changed.

4.3 Validation

Each `trans_clean` step was checked before moving on:

- Row count of `trans_clean` matches `trans`.
- Dates converted with no NULLs and a sensible range.
- Every code received an English label (the check returned nothing).
- Blank values became NULL.
- A sample of rows was reviewed by eye.

5. Dormant Account Analysis

5.1 What is a dormant account

A dormant account is one with no customer activity for more than 90 days, counted back from the reference date of 31 December 1998, the last transaction date in `trans_clean`.

Why 90 days. A longer cutoff, such as 180 days, is the more common choice for calling an account dormant. But with 180 days, too few accounts would qualify, and a group that small is not enough to analyse or compare. A 90-day cutoff gives a group large enough to profile and compare with active accounts. The three dormant bands (90-180, 180-365 and 365+) are still kept in `account_activity`, so the longer cutoffs can be seen inside the group.

5.2 What counts as customer activity

Included: every transaction a customer makes or sets up, such as cash deposits and withdrawals, card withdrawals, transfers, pensions, household and loan payments.

Excluded: three types of entries the bank posts on its own.

Excluded entry	Why
Interest credited	Added by the bank, not by the customer
Payment for statement	A fee charged by the bank
Sanction interest (negative balance)	A penalty charged by the bank

These rows would make an unused account look active, since the bank keeps posting them. Counting them would hide the dormant accounts.

5.3 Flagging dormant accounts: the `account_activity` view

The `account_activity` view gives one row per account with its last customer activity date, days inactive at the reference date, and a status. It is needed because the transaction table has a million rows, but dormancy is decided per account.

It is built in three steps:

1. Keep only customer transactions and rank them newest first within each account (`ROW_NUMBER`).
2. Take the top-ranked row to get each account's last activity date.
3. Join to `account` with a `LEFT JOIN`, so accounts with no customer activity are kept instead of dropped.

Status is set by days inactive:

Status	Rule
Active	90 days or less
Dormant 90-180	More than 90 days, up to 180
Dormant 180-365	More than 180 days, up to 365
Dormant 365+	More than 365 days
Never active	No customer activity at all

5.4 Size of the problem

The `account_percentage` view counts the accounts in each status and gives each as a percentage of all accounts. This shows how large the dormancy problem is and how it splits across the bands.

Status	Accounts	% of accounts
Active	4,479	99.53
Dormant 90-180	5	0.11
Dormant 180-365	8	0.18
Dormant 365+	8	0.18

Observed result. 21 of the 4,500 accounts are dormant, split 5, 8 and 8 across the three bands.

5.5 Dormant vs active grouping: the `account_group` table

All three dormant bands are combined into one `Dormant` group (`grp`), so later queries can compare Dormant with Active. The `v_account_group` view does this, but it was slow to recompute on every query. So the result was saved as a table, `account_group` , with `account_id` as the primary key. This made all later calculations faster.

5.6 Profile: dormant vs active

Why this comparison. This section looks at how dormant accounts differ from active ones, to understand the characteristics of the two kinds of customer. It compares account age and the age of the account holder. It also checks whether they use the bank's

other services (a card, a loan and a standing order) and how the average transaction amount differs between the two groups.

Six views were built, one for each comparison:

	Comparison	View	What it measures
a	Account age	`grp_account_age`	Average, minimum and maximum years the account has been open at the reference date
b	Account owner age	`grp_owner_age`	Average, minimum and maximum age of the account owner (disposition type OWNER) at the reference date
c	Average amount per transaction	`grp_avg_amount`	Number of customer transactions and their average amount
d	Accounts with a card	`has_a_card`	Count and percentage of accounts with a credit card, linked through `disp`
e	Accounts with a loan	`has_a_loan`	Count and percentage of accounts with a loan
f	Accounts with a standing order	`has_a_order`	Count and percentage of accounts with a standing order

Results

Measure	Active	Dormant
Accounts	4,479	21
Account age, average (years)	3.4	3.7
Account age, min to max (years)	1.0 to 6.0	1.6 to 5.9
Owner age, average (years)	44.1	37.7
Owner age, min to max (years)	16 to 80	18 to 60
Customer transactions	714,760	1,037
Average amount per transaction	8,699	9,359
Accounts with a card	892 (19.9%)	0 (0.0%)

Measure	Active	Dormant
Accounts with a loan	680 (15.2%)	2 (9.5%)
Accounts with a standing order	3,753 (83.8%)	5 (23.8%)

Observed result

Of the 4,500 accounts, 4,479 are active and only 21 are dormant, so the dormant group is about 0.5% of all accounts.

- **Account age:** Dormant accounts are slightly older, at 3.7 years on average against 3.4 for active ones. Both groups cover a similar range, from about 1 to 6 years.
- **Account owner age:** Dormant owners are younger, at 37.7 years on average against 44.1 for active owners. The oldest dormant owner is 60, while active owners go up to 80.
- **Average transaction amount:** Dormant accounts have a slightly higher average, at 9,359 against 8,699 for active accounts. The dormant group made only 1,037 customer transactions in total, against 714,760 for active accounts.
- **Card:** None of the 21 dormant accounts has a card. 892 active accounts (19.9%) do, which is every card in the data.
- **Loan:** 2 dormant accounts (9.5%) have a loan, against 680 active accounts (15.2%).
- **Standing order:** 5 dormant accounts (23.8%) have a standing order, against 3,753 active accounts (83.8%).

Overall, dormant accounts use the bank's other services far less than active accounts. The gap is clearest for standing orders and cards. Dormant owners also tend to be younger.

Note. Only 21 of the 4,500 accounts are dormant, so a single account changes the percentages a lot. For example, the 9.5% with a loan is just 2 accounts. The comparison with active accounts is too small to be reliable, so these results show a pattern in this data and shouldn't be taken as a firm rule.

5.7 Behaviour before going quiet

Why the last 6 months. An account usually changes its behaviour in the months before it goes quiet. Looking at the whole history would give a lot of information, but it would mix in the account's normal behaviour and hide that change. So all the views in this section look only at the 6 months up to each account's last customer activity date.

a) Last customer transactions (`dormant_lst_trnx`)

Lists the customer transactions on each dormant account's last active day, with type, operation, amount and balance.

Why this view. I want to see what the last transaction of each dormant account looked like: whether it was a debit or a credit, how much it was, and how much balance was left. This also helps plan the further analysis. Bank-generated entries (interest, statement fees and sanction interest) are left out, so what remains is the last thing the customer actually did.

Account	Status	Last activity	Type	Operation	Category	Amount	Balance
799	Dormant 365+	1996-08-25	Debit	Withdrawal in Cash	-	2,300.00	9,348.30
1720	Dormant 365+	1996-09-07	Debit	Withdrawal in Cash	-	500.00	11,400.00
2029	Dormant 365+	1996-12-09	Debit	Withdrawal in Cash	-	300.00	8,900.00
459	Dormant 365+	1997-03-01	Debit	Withdrawal in Cash	-	700.00	8,100.00
2180	Dormant 365+	1997-03-10	Credit	Credit in Cash	-	900.00	11,300.00
1563	Dormant 365+	1997-08-09	Debit	Withdrawal in Cash	-	3,900.00	10,000.00
869	Dormant 365+	1997-10-08	Debit	Withdrawal in Cash	-	600.00	10,300.00
3814	Dormant 365+	1997-10-17	Debit	Withdrawal in Cash	-	2,100.00	10,800.00
182	Dormant 180-365	1998-01-16	Debit	Withdrawal in Cash	-	2,200.00	10,000.00
3888	Dormant 180-365	1998-01-31	Debit	Withdrawal in Cash	-	1,200.00	9,301.30
1056	Dormant 180-365	1998-02-07	Debit	Outgoing transfer to another bank	Household payment	5,692.00	2,449.10
1430	Dormant 180-365	1998-02-08	Credit	Credit in Cash	-	1,000.00	11,000.00

First 12 rows shown, oldest last activity first.

b) Activity in the 6 months before going quiet (`last_6m_dormant_activity`)

For each dormant account, the number of customer transactions and average amount in the 6 months up to its last activity date.

Why this view. I want to see how active each dormant account was before it stopped: how many transactions it made and the average amount per transaction. A low count would suggest the account was already fading, while a high count would suggest it stopped suddenly.

Account	Status	Last activity	Transactions (last 6 months)	Average amount
182	Dormant 180-365	1998-01-16	5	2,340
459	Dormant 365+	1997-03-01	5	1,740
799	Dormant 365+	1996-08-25	3	2,800
869	Dormant 365+	1997-10-08	7	2,143
1056	Dormant 180-365	1998-02-07	21	14,245
1430	Dormant 180-365	1998-02-08	6	2,533
1563	Dormant 365+	1997-08-09	6	2,117
1589	Dormant 90-180	1998-09-10	3	1,167
1720	Dormant 365+	1996-09-07	6	1,333
1735	Dormant 180-365	1998-03-29	5	1,340
2029	Dormant 365+	1996-12-09	3	567
2180	Dormant 365+	1997-03-10	6	1,433

First 12 accounts shown, by account number.

c) Active vs dormant activity in a 6-month window (`active_vs_dormant_actvity_last_6m`)

Compares average transactions and average amount per account over the last 6 months of activity. Active accounts use the window ending 31 December 1998. Dormant accounts use the window ending on their own last activity date.

Why this view. Views a and b show how dormant accounts behaved on their own, but that doesn't say whether the behaviour was unusual. This view sets it against active accounts to answer that. If dormant accounts were already making fewer or smaller transactions in

their last 6 months, that points to an early warning sign the bank could watch for. If their numbers look the same as active accounts, there was no visible warning before they stopped.

Group	Accounts	Average transactions (last 6 months)	Average amount (last 6 months)
Active	4,479	22.7	8,597
Dormant	21	7.3	4,826

Observed result

- **Transactions:** Dormant accounts averaged 7.3 customer transactions in their last 6 months, against 22.7 for active accounts. That is about a third as many.
- **Average amount:** Dormant accounts averaged 4,826 per transaction, against 8,597 for active accounts. That is about 44% lower.

So dormant accounts were already doing less, and doing smaller transactions, in the months before they went quiet. This suggests a drop in both transaction count and amount could work as an early warning sign.

This also differs from the profile in 5.6, where dormant accounts had a higher average amount over their whole history (9,359 against 8,699). Their amounts were higher over their whole history but lower in the last 6 months, which supports looking at the last 6 months rather than the full history.

d) Debits and credits per dormant account (`each_dormant_no_of_debit_credit`)

For each dormant account, the number of credits and debits (anything that isn't a credit) in the 6 months before its last activity.

Why this view. Counting transactions in view b shows how much an account was used, but not in which direction the money was moving. This view splits that activity into credits (money coming in) and debits (money going out). An account that was mostly debiting before it stopped may have been drawing its money out, while one that was mostly crediting may have simply stopped being topped up. The split for each account shows which of the two patterns the dormant accounts follow, and the next view (e) adds it up for all of them.

Account	Status	Last activity	Credits	Debits	Total
182	Dormant 180-365	1998-01-16	3	2	5
459	Dormant 365+	1997-03-01	1	4	5
799	Dormant 365+	1996-08-25	1	2	3

Account	Status	Last activity	Credits	Debits	Total
869	Dormant 365+	1997-10-08	2	5	7
1056	Dormant 180-365	1998-02-07	5	16	21
1430	Dormant 180-365	1998-02-08	2	4	6
1563	Dormant 365+	1997-08-09	1	5	6
1589	Dormant 90-180	1998-09-10	0	3	3
1720	Dormant 365+	1996-09-07	0	6	6
1735	Dormant 180-365	1998-03-29	1	4	5
2029	Dormant 365+	1996-12-09	1	2	3
2180	Dormant 365+	1997-03-10	2	4	6

First 12 accounts shown, by account number.

e) Debits and credits across all dormant accounts (`last_6m_dormant_total_debit_credit`)

The same count for all dormant accounts together, with the percentage that are credits.

Credits	Debits	% credits
33	121	21.4

Observed result

In the 6 months before they went quiet, the 21 dormant accounts made 154 customer transactions: 33 credits and 121 debits. Only 21.4% were credits, so almost four in five transactions were money going out.

Dormant accounts were mostly drawing money out rather than being topped up before they stopped. Together with view c, where transaction count and amount were both falling, this fits accounts that were being run down before they went quiet.

6. SQL Techniques Used

Loading and preparing data. The large `trans` table was loaded with `LOAD DATA LOCAL INFILE`, which is much faster than the import wizard for over a million rows. The cleaned copies were built with `CREATE TABLE ... AS SELECT`, so the raw `trans` table stayed untouched. The other tables were changed in place with `ALTER TABLE` and `UPDATE`, adding new columns and dropping the old ones.

Cleaning and transforming. `STR_TO_DATE` and `CAST` turned the YYMMDD numbers into real dates. `TRIM` and `NULLIF` converted blank values to NULL. `CASE WHEN` translated the Czech codes into English labels. The client `birth_number` needed string functions (`LEFT`, `RIGHT`, `SUBSTRING`, `CONCAT` and `LPAD`) to pull out the year, month and day. For women the month has 50 added, so 50 was subtracted before building the date.

Finding the last customer activity. This is the core of the analysis. A CTE (`WITH`) split the logic into steps: first keep only customer transactions, then find each account's latest one, then calculate the days inactive. The window function `ROW_NUMBER() OVER (PARTITION BY account_id ORDER BY trans_date DESC)` ranked each account's customer transactions from newest to oldest, so the row ranked 1 is the last activity. A `LEFT JOIN` from `account` kept the accounts with no customer activity, which an inner join would have dropped. `DATEDIFF` then measured the days inactive at the reference date, and `CASE WHEN` set the status bands.

Filtering out bank-generated entries. Every customer-activity filter uses `IS NULL OR NOT IN (...)`. The `IS NULL` part matters, because `NOT IN` alone drops rows where `k_symbol_en` is NULL, and those are normal customer transactions.

Profiling and comparing. Correlated `EXISTS` subqueries checked whether each account has a card, a loan or a standing order. Subqueries also gave the total for the percentages and the two-step average in the 6-month comparison. `DATE_SUB` set the 6-month windows, and `TIMESTAMPDIFF` calculated account and owner ages. Conditional aggregation, such as `SUM(type_en = 'Credit')`, counted credits and debits in a single pass. `GROUP BY` and `ORDER BY FIELD()` summarised the statuses in a fixed order.

Performance and structure. A composite index on `trans_clean (account_id, trans_date)` sped up every query on the large table. Each analysis step is a view, so results are reusable and easy to re-run. The `account_group` view was slow to recompute, so it was saved as a table with `account_id` as the primary key, which made the later queries faster.

7. Final Results

Of the 4,500 accounts in the Czech Bank data, 21 were dormant, meaning they had no customer activity for more than 90 days as of 31 December 1998. The other 4,479 were active. The analysis set out to understand how these 21 accounts differ from the active ones and what they looked like before they went quiet.

Profile: dormant vs active. The clearest difference is in the use of the bank's other services. Dormant accounts used far fewer of them than active accounts. None of the 21 has a card, 9.5% have a loan against 15.2% of active accounts, and 23.8% have a standing

order against 83.8%. Dormant owners were also younger on average (37.7 years against 44.1). Account age and average transaction amount over the whole history did not separate the two groups much.

Behaviour in the last 6 months. The strongest findings came from the 6 months before each account went quiet.

Dormant accounts averaged 7.3 customer transactions in that window, against 22.7 for active accounts, and their average transaction amount was 4,826, against 8,597. Across the dormant accounts, only 21.4% of transactions were credits (33 credits against 121 debits).

Together, these suggest accounts that are about to go dormant make fewer and smaller transactions and are mostly drawn down. A bank could watch for this pattern as an early warning.

Limitation. The dormant group is only 21 accounts, so a single account changes the percentages a lot. These results show a pattern in this data and shouldn't be taken as a firm rule.